

Tetra-SoC Project Plan

The aim of this project is to tape-out an soc with a simple 4-bit cpu subsystem on the TTSKY26B TinyTapeout shuttle.

Block Diagram

TODO

Hierrchy Diagram

TODO

CPU block diagram

TODO

System Memory Map

TODO

Boot Flow

1. Core Released from Reset, Reset Vector is in Boot ROM

Reset vector is 0x4000

Boot ROM, configure SPI to read in batch mode for the predefined metadata header, and then afterwards it continues to read the first chunk of data.

Boot ROM, contains the decryption key for

CPU hands over control to debugger, gets stuck in NOP loop until the debugger pulls it into the boot ROM address.

2. SPI Master (Metadata Fetch from Emulated Flash)

The SPI mem control unit Interfaces with emulated 23LC512 SPI RAM chip by Mike Bell ([spi ram emu](#))

Datasheet: [23LC512 SPI RAM chip datasheet](#)

Read (0x3), Write (0x2) and Fast Read (0x7) are supported.

Key Details

This Provides 64KB (16-bit address) of memory to use for storing programs and/or data. Docs: <https://github.com/MichaelBell/spi-ram-emu>

We have an implementation for this already ([here](#)), It may need updates + conversion into a wishbone peripheral that can be programmed by the boot ROM.

Limitations

QUITE SLOW - Fetch time for a program = $24 + 8 \cdot \text{NUM_BYTES}$ (normal read)

The reads shall be aligned so it does not need to implement fast mode.

cs must be held high for ~ 50 cycles in between reads.

At 125MHz RP2040 clk, we get a max SCLK speed of 15MHz (aim for 12.5MHz?)

3. Program Fetch + Decryption

After data is fetched by the SPI, there will be some metadata (encrypted? program length) This configuration is passed into the crypto subsystem and then decrypted if necessary. The data is streamed into a program buffer that

3. Program Execution

The CPU executes programs from the instruction fetch buffer, and writes data back to RAM.

Do we include a small internal RAM for variables and intermediate data/programs? (how small is small?)

How do branches work when they go back to code that has been unloaded???

Do we need a simple Memory Management Unit to flag this up and refetch the required page? This would make some branches really slow and others seamless.

Can we jump to the fetch address in the SPI memory controller, to just load the data that has been lost back in?

CPU Design/Testing Tasks

 Assignee	 Title	 Due Date	 Status
Aiva Cooney-...	Finalize the instruction set and design opcode.	 Date	In progress ▾
Pengfei Xiao	Design the RAM system (for memory access instructions	23 Apr 2026	Not started ▾
Thomas Cowie	Add LOAD (LDA,LDB) all addressing modes.	23 Apr 2026	Not started ▾
Thomas Cowie	Design Control Unit	 Date	In progress ▾
Aiva Cooney-...	Add Compare operations to ALU	 Date	In progress ▾
Aiva Cooney-...	Add arithmetic Instructions to ALU	 Date	In progress ▾
Tristan K Villa...	Design the JTAG TAG and cpu hardware debug modules	 Date	Not started ▾
James Ashie ...	Write Test Programs using the ISA	 Date	Not started ▾
Cameron Sp...	CPU Block Diagram	 Date	Not started ▾

Software Tasks

 Assignee	 Title	 Due Date	 Status
James Ashie ...	Write an Assembler	26 Apr 2026	In progress ▾
James Ashie ...	Write a simpl	26 Apr 2026	Completed ▾
James Ashie ...	Design/Test SPI Memory Controller	30 Apr 2026	In progress ▾
James Ashie ...	Design/Test UART peripheral	30 Apr 2026	In progress ▾
James Ashie ...	Design Wishbone Bus		
 Person	Dig Top Integration	 Date	

Tetra SoC Design/Testing Tasks

 Assignee	 Title	 Due Date	 Status
James Ashie ...	Improve Doc	26 Apr 2026	In progress ▾
James Ashie ...	Create Repos and move over files from super-simple-spi-cpu	26 Apr 2026	Completed ▾
James Ashie ...	Design/Test SPI Memory Controller	30 Apr 2026	In progress ▾
James Ashie ...	Design/Test UART peripheral	30 Apr 2026	In progress ▾
James Ashie ...	Design Wishbone Bus		
 Person	Dig Top Integration	 Date	